

Java Programming

What is Programming?

Programming is the process of writing instructions for a computer to perform a particular task.

A computer does not understand human language directly. We write instructions using a programming language, and the computer processes those instructions.

Example:-

Suppose we want a computer to calculate the sum of two numbers.

Take two numbers → add them → display the result.

JAVA Code:-

```
int a = 10;  
int b = 20;  
int sum = a + b;  
System.out.println(sum);
```

What is Java?

Java is a high-level, object-oriented, general-purpose programming language used to develop software applications.

Java allows developers to write instructions that computers can execute.

Think of Java as a language used to communicate instructions to a computer.

```
System.out.println("Welcome to Java");
```

(this tells as a instruction to computer that print on the screen : “Welcometo Java”)

Introduction to Java:-

History of Java

- Java was originally developed at Sun Microsystems.
- The project started in the early 1990s.
- James Gosling and his team played a major role in its development.
- Java was officially released in 1995.
- Sun Microsystems was acquired by Oracle in 2010.
- Java is now maintained and developed under Oracle and the broader Java community.

Original Purpose

Java was designed with the goal of creating software that could work across different computer platforms.

This led to the famous idea:

Write Once, Run Anywhere (WORA)

Why is Java Called Platform Independent?

This is one of the most important concepts for beginners.

Normally, a program written for one operating system may not directly work on another operating system.

Java solves this using the JVM.

Java execution process:-

Java Source Code



Java Compiler



Bytecode (.class)



JVM



Operating System

EX:-

```
System.out.println("Hello");
```

The Java compiler converts the source code into bytecode.

The JVM then executes that bytecode on the particular operating system.

What is Java Programming?

Java programming is the process of designing and writing instructions using Java syntax to solve problems and develop applications.

Java programming involves:

1. Understanding the problem
2. Designing the logic
3. Writing Java code
4. Compiling the code
5. Running the program
6. Testing the result
7. Fixing errors

Example Problem:

Check whether a person is eligible to vote.

Logic:

Take age

↓

Is age ≥ 18 ?

↓

Yes → Eligible

No → Not Eligible

Java:

```
int age = 20;
if (age >= 18) {
    System.out.println("Eligible to vote");
} else
{
    System.out.println("Not eligible");
}
```

Java in Android Development:-

For your students, explain this connection clearly.

Suppose we want to create a Greeting App.

The Android application has:

XML

Used to design the user interface.

Button

EditText

TextView

ImageView

Java

Used to implement the application's behavior.

For example:

User enters name



Clicks Greet



Java gets the name



Java processes it



TextView displays greeting

Example:

```
String name = editText.getText().toString();
```

```
textView.setText("Hello, " + name);
```

Java Fundamentals

A – Variables

A variable is a named memory location used to store a value that can be used by a program.

Ex. `int age = 20;`

`int` → Data Type

`age` → Variable

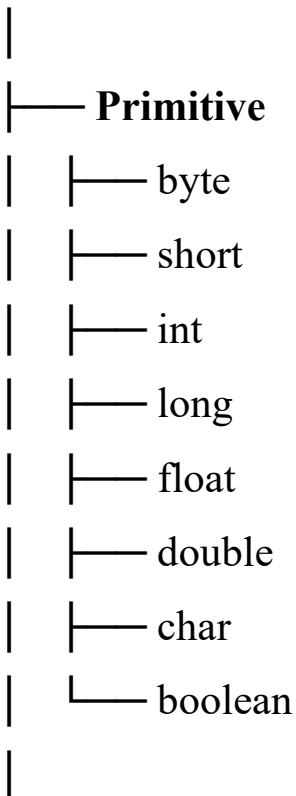
`20` → Value

Ex:- `String name = "Sneha";`

B – Data Types

A data type specifies what kind of value a variable can store.

Data Types



└─ Reference Types

└─ String

└─ Arrays

└─ Classes

└─ Objects

Examples:-

```
int age = 20;
```

```
double price = 99.50;
```

```
char grade = 'A';
```

```
boolean isActive = true;
```

```
String name = "Sneha";
```

C – Operators

Operators are symbols used to perform operations on values and variables.

Arithmetic Operators

+ - * / %

EX:-

```
int a = 10;
```

```
int b = 3;
```

```
System.out.println(a + b);
```

```
System.out.println(a - b);
```

```
System.out.println(a * b);
```

```
System.out.println(a / b);  
System.out.println(a % b);
```

D – Input / Output

Output

Output means displaying information.

Ex:-

```
System.out.println("Hello");
```

Input

Input means accepting information from the user.

Ex:-

```
Scanner sc = new Scanner(System.in);  
System.out.print("Enter your name: ");  
String name = sc.nextLine();  
System.out.println("Hello " + name);
```

E – Conditional Statements

Conditional statements are used to make decisions based on conditions.

Example:

```
int marks = 75;  
if (marks >= 40) {  
    System.out.println("Pass");  
} else {
```

```
System.out.println("Fail");  
}
```

Types:-

1. if
2. if-else
3. else-if
4. Nested if
5. switch

F – Loops

A loop is used to repeatedly execute a block of code while a specified condition or iteration rule is satisfied.

Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

1
2
3
4
5

Types of loops

for

while

do-while

G – Functions / Methods

Java generally calls functions methods.

A method is a block of code defined inside a class that performs a specific task and can be called when needed.

Example:

```
static void greet() {  
    System.out.println("Hello");  
}
```

Calling it:

```
greet();
```

Output:

Hello

Why use methods?

Instead of writing the same code repeatedly:

Write once

↓

Create method

↓

Call whenever required

H – Arrays

An array is a fixed-size collection of elements of the same data type stored under one variable name.

Example:

```
int[] marks = {80, 75, 90, 85};
```

marks

|— [0] → 80

|— [1] → 75

|— [2] → 90

|— [3] → 85

Important:

Java array indexing starts from 0.

Therefore:

```
System.out.println(marks[0]);
```

Output:

80

I – OOP Concepts

Java is based on object-oriented programming.

1. Class

A class is a blueprint or template used to define the data and behavior of objects.

Example:

```
class Student {  
    String name;  
    int age;  
}
```

2. Object

An object is an instance of a class.

Example:

```
Student s1 = new Student();
```

Here:

Student → Class

s1 → Object reference

new → Creates an object

3. Encapsulation

Encapsulation means bundling data and methods together and controlling access to the data.

Example:

```
class Student {
```

```
private int age;
```

```
    public void setAge(int age) {  
        this.age = age;  
    }  
    public int getAge() {  
        return age;  
    }  
}
```

4. Inheritance

Inheritance allows one class to acquire properties and behavior from another class.

Example:

```
class Animal { void eat() {  
    System.out.println("Eating");  
}  
}  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

Dog inherits from Animal.

5. Polymorphism

Polymorphism means allowing the same interface or method name to represent different behaviors depending on the situation.

A simple example is method overloading:

```
void add(int a, int b) {  
    System.out.println(a + b);  
}  
  
void add(int a, int b, int c) {  
    System.out.println(a + b + c);  
}
```

Same method name:

add()

but different parameters.

6. Abstraction

Abstraction means hiding unnecessary implementation details and exposing only the important functionality.

example, when using an ATM:

User

↓

Select Withdraw

↓

Enter Amount



Money Received